EXP 6	Implementation of two input sine function.

AIM: Understand the concept of implementation of two input sine function using Genetic algorithm.

Algorithm:

1. Define the fitness function f(x,y)=sin(x)+sin(y)
2. Initialize the population with random values
3. Apply genetic operations (selection, crossover, mutation)
4. Run the genetic algorithm for a fixed number of generations
5. Print the final best solution found by the genetic algorithm.

PROGRAM

```
import random
import math

# Define the fitness function (sine function with two inputs)
def fitness_function(x, y):
    return math.sin(x) + math.sin(y)

# Initialize the population
def initialize_population(pop_size, lower_bound, upper_bound):
    return [(random.uniform(lower_bound, upper_bound), random.uniform(lower_bound,
upper_bound)) for _ in range(pop_size)]

# Select parents based on their fitness
def select_parents(population):
    total_fitness = sum(fitness_function(x, y) for x, y in population)
    roulette_wheel = [fitness_function(x, y) / total_fitness for x, y in population]
    parent1 = random.choices(population, weights=roulette_wheel)[0]
    parent2 = random.choices(population, weights=roulette_wheel)[0]
    return parent1, parent2

# Perform crossover to create a new generation
def crossover(parent1, parent2, crossover_prob=0.7):
    if random.random() < crossover_prob:
        crossover_point = random.randint(0, 1)
        child1 = (parent1[0], parent2[1])
        child2 = (parent2[0], parent1[1])
        return child1, child2
    else:
        return parent1, parent2
```

```
# Perform mutation in the population
def mutate(individual, mutation_prob=0.01):
    x, y = individual
    if random.random() < mutation_prob:
        x += random.uniform(-0.1, 0.1)
    if random.random() < mutation_prob:
        y += random.uniform(-0.1, 0.1)
    return x, y

# Genetic Algorithm
def genetic_algorithm(generations, pop_size, lower_bound, upper_bound):
    population = initialize_population(pop_size, lower_bound, upper_bound)

    for gen in range(generations):
        new_population = []

        while len(new_population) < pop_size:
            parent1, parent2 = select_parents(population)
            child1, child2 = crossover(parent1, parent2)
            child1 = mutate(child1)
            child2 = mutate(child2)
            new_population.extend([child1, child2])

        population = new_population

        best_individual = max(population, key=lambda ind: fitness_function(*ind))
        print(f"Generation {gen+1}: Best individual - {best_individual}, Fitness -
{fitness_function(*best_individual)}")

    return max(population, key=lambda ind: fitness_function(*ind))

if __name___== "__main__":
    generations = 20
    pop_size = 100
    lower_bound = -2 * math.pi
    upper_bound = 2 * math.pi

    best_solution = genetic_algorithm(generations, pop_size, lower_bound, upper_bound)
    print(f"Best solution found: {best_solution}, Fitness: {fitness_function(*best_solution)}")
```

Output:

```
Generation 1: Best individual - (1.5777504712208659, -5.172491976708219), Fitness -
1.8959825877850767
Generation 2: Best individual - (1.5777504712208659, -5.172491976708219), Fitness -
1.8959825877850767
Generation 3: Best individual - (1.5777504712208659, -5.172491976708219), Fitness -
```

1.8959825877850767
Generation 4: Best individual - (1.5777504712208659, -4.9160074310511375), Fitness - 1.9793171081830292
Generation 5: Best individual - (-5.115695862433251, -4.9160074310511375), Fitness - 1.899109487859969
Generation 6: Best individual - (1.5777504712208659, -4.9160074310511375), Fitness - 1.9793171081830292
Generation 7: Best individual - (1.5777504712208659, -4.9160074310511375), Fitness - 1.9793171081830292
Generation 8: Best individual - (1.5777504712208659, -4.9160074310511375), Fitness - 1.9793171081830292
Generation 9: Best individual - (1.5777504712208659, -4.9160074310511375), Fitness - 1.9793171081830292
Generation 10: Best individual - (1.5777504712208659, -4.252686927315365), Fitness - 1.8961605507621613
Generation 11: Best individual - (1.5777504712208659, -4.9160074310511375), Fitness - 1.9793171081830292
Generation 12: Best individual - (1.5777504712208659, -4.9160074310511375), Fitness - 1.9793171081830292
Generation 13: Best individual - (1.5777504712208659, -4.252686927315365), Fitness - 1.8961605507621613
Generation 14: Best individual - (1.549677484824824, -4.9160074310511375), Fitness - 1.979118293693101
Generation 15: Best individual - (1.5777504712208659, -4.9160074310511375), Fitness - 1.9793171081830292
Generation 16: Best individual - (1.549677484824824, -4.9160074310511375), Fitness - 1.979118293693101
Generation 17: Best individual - (1.5777504712208659, -4.9160074310511375), Fitness - 1.9793171081830292
Generation 18: Best individual - (1.5777504712208659, -4.252686927315365), Fitness - 1.8961605507621613
Generation 19: Best individual - (1.5777504712208659, -4.952045490050521), Fitness - 1.9713953861381543
Generation 20: Best individual - (1.5777504712208659, -4.9160074310511375), Fitness - 1.9793171081830292
Best solution found: (1.5777504712208659, -4.9160074310511375), Fitness: 1.9793171081830292

Result: Thus the above program implementation of two input sine function using genetic algorithm executed successfully.